Task 1

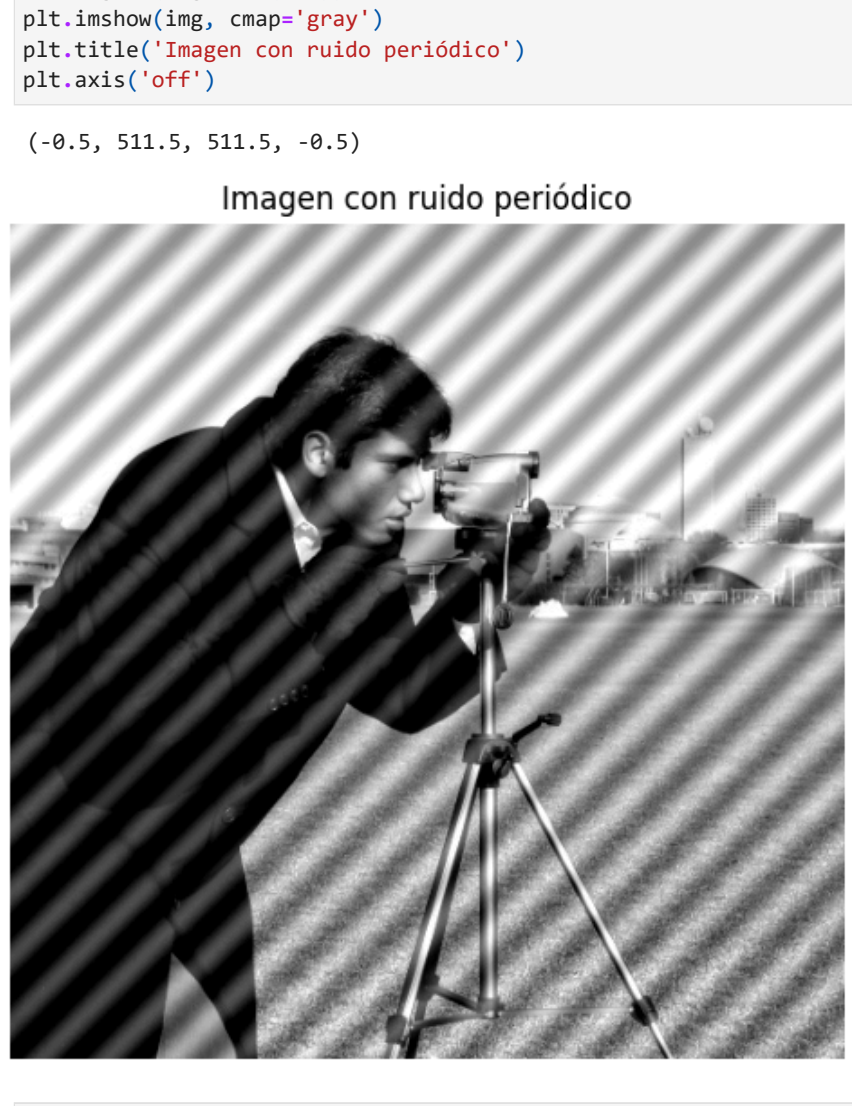
Usted trabaja para una empresa de imágenes satelitales. Una de las cámaras en órbita tiene una interferencia electrónica que genera un ruido sinusoidal (patrón de rayas diagonales) sobre las fotografías de la superficie terrestre. Los filtros espaciales tradicionales (Gaussian Blur) destruyen los detalles geográficos necesarios. Por ello usted ha decidido idear una solución tomando como base fotografías que tiene a mano con el mismo problema previo a implementar la solución real. Con esto en mente, realice:

1. Cargue la imagen periodic_noise.jpg en escala de grises.
2. Calcule la Transformada Discreta de Fourier (DFT) y desplace el componente de frecuencia cero al centro.
3. Muestre el Espectro de Magnitud en escala logarítmica.
4. Identifique visualmente los "picos" de energía que no corresponden a la información natural de la imagen (puntos brillantes fuera del centro).
5. Cree una máscara (Notch Filter) que bloquee específicamente esas frecuencias parásitas (haciéndolas cero), pero preserve el resto del espectro, incluyendo el componente DC.
6. Aplique la Transformada Inversa (IDFT) para recuperar la imagen espacial. Para esta parte se espera que entregue: • Código funcional y visualización correcta del espectro con los picos de ruidos señalados • Imagen restaurada exitosamente (sin rayas y con detalles nítidos) • Explique por qué un filtro de promedio (average filter) de 5x5 en el dominio espacial hubiera sido una mala solución para este problema especifico

In [65]:
```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

In [66]:
```python
img = cv2.imread('periodic_noise.jpg', cv2.IMREAD_GRAYSCALE)

plt.figure(figsize=(6,6))
plt.imshow(img, cmap='gray')
plt.title('Imagen con ruido periódico')
plt.axis('off')
```

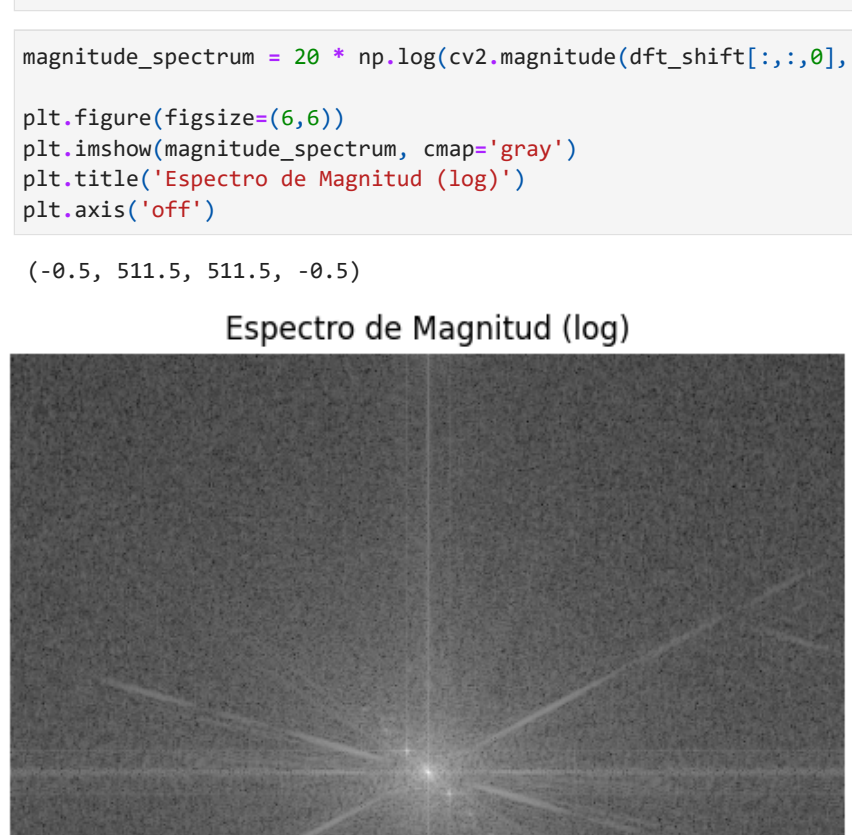
Out[66]:
```
(-0.5, 511.5, 511.5, -0.5)
```



In [ ]:
```python
dft = cv2.dft(np.float32(img), flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft)  # Mover frecuencia cero, centrada
```

In [68]:
```python
magnitude_spectrum = 20 * np.log(cv2.magnitude(dft_shift[:,:,0], dft_shift[:,:,1]) + 1)

plt.figure(figsize=(6,6))
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title('Espectro de Magnitud (log)')
plt.axis('off')
```

Out[68]:
```
(-0.5, 511.5, 511.5, -0.5)
```



se pueden observar los picos de luz tirando en forma de estrella

In [69]:
```python
plt.imsave('espectro.png', magnitude_spectrum, cmap='gray')
```

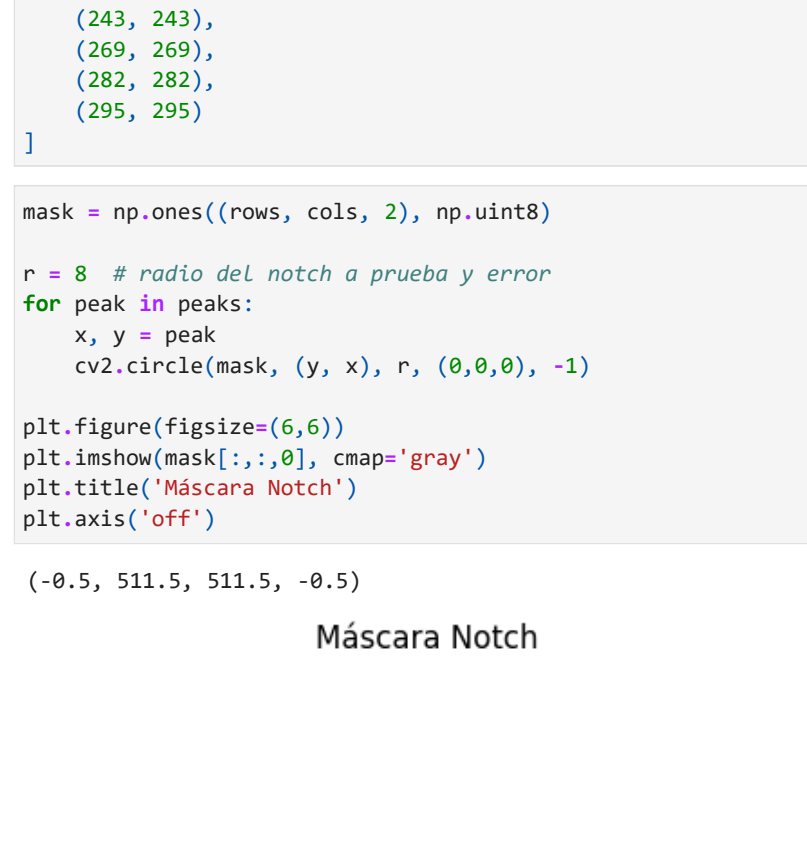
se guardo para usar con ayuda de paint las coordenadas de los picos y con eso logramos definir que los picos son:

(218, 218), (230, 230), (243, 243), (269, 269), (282, 282), (295, 295)

In [70]:
```python
rows, cols = img.shape
peaks = [
    (218, 218),
    (230, 230),
    (243, 243),
    (269, 269),
    (282, 282),
    (295, 295)
]
```

In [71]:
```python
mask = np.ones((rows, cols, 2), np.uint8)

r = 8  # radio del notch a prueba y error
for peak in peaks:
    x, y = peak
    cv2.circle(mask, (y, x), r, (0,0,0), -1)

plt.figure(figsize=(6,6))
plt.imshow(mask[:,:,0], cmap='gray')
plt.title('Máscara Notch')
plt.axis('off')
```

Out[71]:
```
(-0.5, 511.5, 511.5, -0.5)
```



In [72]:
```python
fshift = dft_shift * mask

# Volver a dominio espacial
f_ishift = np.fft.ifftshift(fshift)
img_back = cv2.idft(f_ishift)
img_back = cv2.magnitude(img_back[:,:,0], img_back[:,:,1])

# Normalizar para visualizar
img_back = cv2.normalize(img_back, None, 0, 255, cv2.NORM_MINMAX)
img_back = np.uint8(img_back)

plt.figure(figsize=(6,6))
plt.imshow(img_back, cmap='gray')
plt.title('Imagen Restaurada (sin ruido periódico)')
plt.axis('off')
```

Out[72]:
```
(-0.5, 511.5, 511.5, -0.5)
```

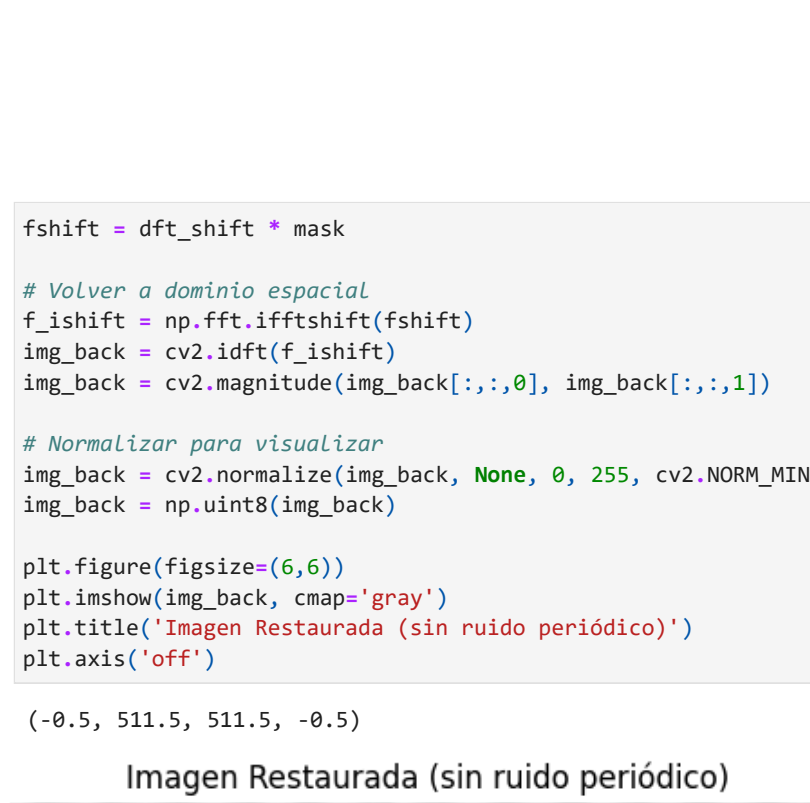


Imagen Restaurada (sin ruido periódico)

por que un filtro de promedio de 5x5 hubiera sido una mala solucion?

El filtro de promedio 5×5 reemplaza cada pixel por el valor promedio de sus vecinos, lo que produce un efecto de suavizado general en la imagen. Sin embargo, el ruido presente en este caso no era aleatorio, sino periódico y con una frecuencia bien definida.

Entonces el filtro no habría eliminado las líneas de ruido, sino que únicamente habría reducido de forma general las variaciones de intensidad en la imagen. Como este tipo de filtro tiene su respuesta máxima en las bajas frecuencias y atenúa progresivamente las altas, el resultado visual habría sido una imagen más borrosa, donde los detalles geográficos importantes se degradan, mientras que el patrón de ruido todavía sería perceptible.

En resumen, se habría perdido información útil de la imagen sin eliminar eficazmente la interferencia sinusoidal, lo que lo convierte en una solución inadecuada para este problema específico.

Task 2

In [2]:
```python
import cv2
import numpy as np

# ----------------------------
# 1) Cargar imagen y asegurar binaria
# ----------------------------
path = "fingerprint_noisy.png"
img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError(f"No se pudo leer {path}. Verifica la ruta.")

# Otsu -> binaria (0/255)
_, bw = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Heurística: si quedó "muy blanco", invertir
# (en general conviene que la huella/crestas sea el foreground blanco)
if np.mean(bw == 255) > 0.5:
    bw = cv2.bitwise_not(bw)

# ----------------------------
# 2) Quitar ruido sal (puntitos blancos)
#    -> Apertura (opening) = erosión + dilatación
# ----------------------------
k1 = 3  # tamaño kernel (3, 5...). 3 suele ser buen inicio
se1 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k1, k1))
step2 = cv2.morphologyEx(bw, cv2.MORPH_OPEN, se1, iterations=1)

# ----------------------------
# 3) Conectar grietas
#    -> Cierre (closing) = dilatación + erosión
# ----------------------------
k2 = 5  # tamaño kernel para cerrar grietas (5, 7...). Depende de la imagen
se2 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k2, k2))
final_img = cv2.morphologyEx(step2, cv2.MORPH_CLOSE, se2, iterations=1)

# ----------------------------
# Demostración: orden inverso (primero cerrar, luego abrir)
# ----------------------------
inverse = cv2.morphologyEx(
    cv2.morphologyEx(bw, cv2.MORPH_CLOSE, se2, iterations=1),
    cv2.MORPH_OPEN, se1, iterations=1
)

# ----------------------------
# Mostrar resultados (sin matplotlib): OpenCV en notebook
# (Si esto no te muestra bien en tu entorno, abajo te dejo alternativa)
# ----------------------------
try:
    from IPython.display import display
    # Convertir a RGB para que se vea bien si tu notebook lo requiere
    display(cv2.cvtColor(bw, cv2.COLOR_GRAY2RGB))
    display(cv2.cvtColor(step2, cv2.COLOR_GRAY2RGB))
    display(cv2.cvtColor(final_img, cv2.COLOR_GRAY2RGB))
    display(cv2.cvtColor(inverse, cv2.COLOR_GRAY2RGB))
except Exception:
    # Alternativa: guarda archivos y ábrelos desde el explorador del notebook
    cv2.imwrite("01_original_binaria.png", bw)
    cv2.imwrite("02_paso2_opening.png", step2)
    cv2.imwrite("03_final_opening_close.png", final_img)
    cv2.imwrite("04_inverso_close_open.png", inverse)
    print("Listo. Guardé 4 imágenes PNG en el directorio actual.")

# Métrica rápida para comparar (opcional)
print("Proporción de pixeles blancos (foreground):")
print("  Original:", np.mean(bw == 255))
print("  Paso 2:", np.mean(step2 == 255))
print("  Final:", np.mean(final_img == 255))
print("  Inverso:", np.mean(inverse == 255))
```

```
array([[[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [  0,   0,   0]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       ...,

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [  0,   0,   0],
        [  0,   0,   0]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [  0,   0,   0],
        [255, 255, 255],
        [  0,   0,   0]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]]], dtype=uint8)
array([[[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [  0,   0,   0]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       ...,

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [  0,   0,   0],
        [  0,   0,   0],
        [  0,   0,   0]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [  0,   0,   0],
        [255, 255, 255],
        [  0,   0,   0]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]]], dtype=uint8)
array([[[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       ...,

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [  0,   0,   0],
        [  0,   0,   0],
        [  0,   0,   0]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [  0,   0,   0],
        [255, 255, 255],
        [  0,   0,   0]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]]], dtype=uint8)
array([[[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       ...,

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [  0,   0,   0],
        [  0,   0,   0]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]],

       [[255, 255, 255],
        [255, 255, 255],
        [255, 255, 255],
        ...,
        [255, 255, 255],
        [255, 255, 255],
        [255, 255, 255]]], dtype=uint8)
Proporción de pixeles blancos (foreground):
  Original: 0.492462158203125
  Paso 2: 0.4240264892578125
  Final: 0.4740409851074219
  Inverso: 0.7629890441894531
```

¿El orden de los factores altera el producto?

Sí. En morfología matemática, **el orden de las operaciones altera el resultado**, porque **Apertura (Opening)** y **Cierre (Closing)** no son conmutativas.

¿Qué pasa con el orden recomendado: Apertura → Cierre?

1) Apertura (Opening)

- **Objetivo:** eliminar el **ruido sal** (puntitos blancos aislados en los valles).
- La apertura equivale a **Erosión → Dilatación**, por lo que:
  - elimina objetos blancos pequeños que no "sobreviven" a la erosión,
  - y luego recupera el tamaño general de las crestas con la dilatación.

2) Cierre (Closing)

- **Objetivo:** conectar **grietas** (pequeñas roturas en las crestas).
- El cierre equivale a **Dilatación → Erosión**, por lo que:
  - la dilatación "puentea" pequeños cortes,
  - la erosión posterior evita que se engorde demasiado.

Resultado típico: **menos ruido blanco** + **crestas más continuas**.

¿Qué hubiera pasado en orden inverso: Cierre → Apertura?

1) Primero Cierre (Closing)

- La dilatación inicial puede:
  - unir segmentos cercanos (bien, porque cierra grietas),
  - pero también puede **engordar** crestas y **cerrar valles**, uniendo regiones que no deberían unirse si el kernel es algo grande.

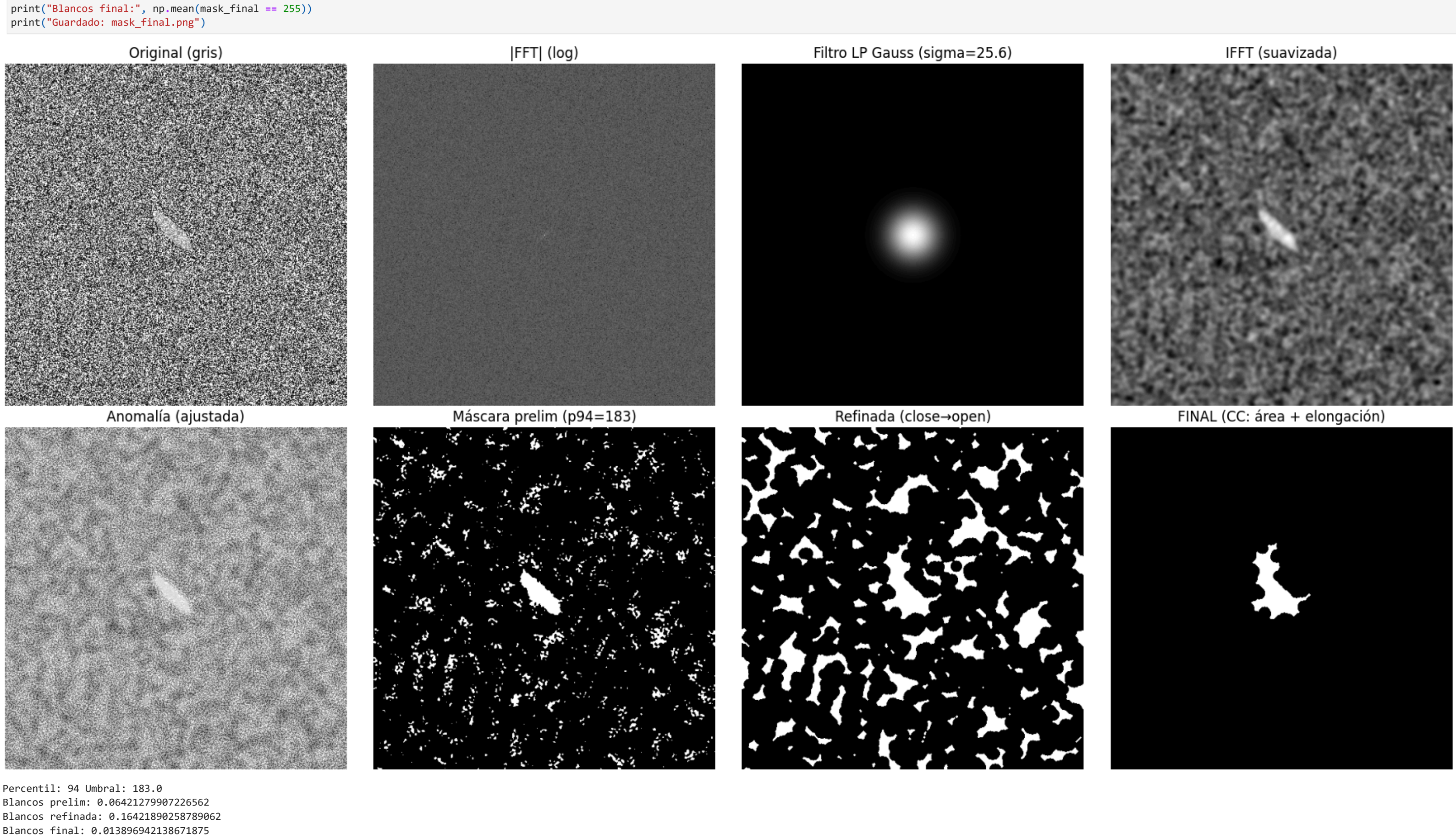
2) Luego Apertura (Opening)

- La apertura elimina "detalles pequeños" del foreground,
- pero **no revierte fácilmente** uniones grandes creadas por el cierre.
- Por eso, es común que el resultado final quede:
  - **más blanco/grueso**,
  - con **pérdida de detalles finos**,
  - y una huella menos útil para matching.

Resultado típico: **sobre-conexión** + **pérdida de estructura**.

Task 3

In [9]:
```python
# Pipeline completo: FFT -> Filtro -> IFFT -> Anomalía -> Threshold -> Morfología -> CC
# (Listo para correr en 1 sola celda)

import cv2
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# 0) Cargar imagen
# ============================================================
path = "textile_defect.jpg"
img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError(f"No se pudo leer {path}. Verifica que esté en el folder del notebook.")

h, w = img.shape
img_f = img.astype(np.float32) / 255.0

# ============================================================
# 1) FFT + filtro low-pass gaussiano (supresión de textura)
# ============================================================
F = np.fft.fft2(img_f)
Fshift = np.fft.fftshift(F)
mag = np.log1p(np.abs(Fshift))

cy, cx = h // 2, w // 2
Y, X = np.ogrid[:h, :w]
dist2 = (Y - cy) ** 2 + (X - cx) ** 2

# Sigma (más pequeño = más agresivo contra textura/grano)
sigma = 0.05 * min(h, w)   # prueba 0.04–0.07 si necesitas
lp = np.exp(-dist2 / (2 * (sigma ** 2))).astype(np.float32)

F_filt = Fshift * lp
smooth = np.fft.ifft2(np.fft.ifftshift(F_filt))
smooth = np.real(smooth)
smooth = cv2.normalize(smooth, None, 0, 1, cv2.NORM_MINMAX)

# ============================================================
# 2) Mapa de anomalía (resalta defecto vs fondo suavizado)
# ============================================================
anomaly = np.abs(img_f - smooth)
anomaly = cv2.normalize(anomaly, None, 0, 1, cv2.NORM_MINMAX)

# Si el defecto aparece "oscuro" en el mapa, invertimos para que sea brillante
roi = anomaly[h//3:2*h//3, w//3:2*w//3]
if roi.min() < roi.mean() - 0.15:
    anomaly_use = 1.0 - anomaly
else:
    anomaly_use = anomaly

an_u8 = (anomaly_use * 255).astype(np.uint8)

# Suavizar anomalía para que el defecto quede más conectado
an_blur = cv2.GaussianBlur(an_u8, (7, 7), 0)

# ============================================================
# 3) Thresholding (segmentación)
# ============================================================
# Percentil menos estricto que p97 (para no quedarnos solo con puntitos)
p = 94  # prueba 92–95 si necesitas
t = np.percentile(an_blur, p)
mask_pre = (an_blur >= t).astype(np.uint8) * 255

# ============================================================
# 4) Morfología (refinamiento)
#    - Primero CIERRE para conectar la rasgadura
#    - Luego APERTURA para limpiar puntos residuales
# ============================================================
se_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (17, 17))  # prueba 13–21
se_open  = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))    # prueba 3–5

mask = cv2.morphologyEx(mask_pre, cv2.MORPH_CLOSE, se_close, iterations=1)
mask = cv2.morphologyEx(mask,     cv2.MORPH_OPEN,  se_open,  iterations=1)

# ============================================================
# 5) Connected Components: escoger región grande + alargada
# ============================================================
num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(mask, connectivity=8)

mask_final = np.zeros_like(mask)
best_score = -1
best_label = None

min_area = 150  # ajusta 80–400 según tamaño esperado del defecto

for i in range(1, num_labels):
    x, y, ww, hh, area = stats[i]
    if area < min_area:
        continue
    aspect = max(ww, hh) / (min(ww, hh) + 1e-6)  # elongación
    score = area * aspect
    if score > best_score:
        best_score = score
        best_label = i

if best_label is not None:
    mask_final[labels == best_label] = 255

cv2.imwrite("mask_final.png", mask_final)

# ============================================================
# 6) Visualización (entregables)
# ============================================================
fig, ax = plt.subplots(2, 4, figsize=(18, 9))
for a in ax.ravel():
    a.axis("off")

ax[0,0].imshow(img, cmap="gray")
ax[0,0].set_title("Original (gris)")

ax[0,1].imshow(mag, cmap="gray")
ax[0,1].set_title("|FFT| (log)")

ax[0,2].imshow(lp, cmap="gray")
ax[0,2].set_title(f"Filtro LP Gauss (sigma={sigma:.1f})")

ax[0,3].imshow(smooth, cmap="gray")
ax[0,3].set_title("IFFT (suavizada)")

ax[1,0].imshow(anomaly_use, cmap="gray")
ax[1,0].set_title("Anomalía (ajustada)")

ax[1,1].imshow(mask_pre, cmap="gray")
ax[1,1].set_title(f"Máscara prelim (p{p}={t:.0f})")

ax[1,2].imshow(mask, cmap="gray")
ax[1,2].set_title("Refinada (close→open)")

ax[1,3].imshow(mask_final, cmap="gray")
ax[1,3].set_title("FINAL (CC: área + elongación)")

plt.tight_layout()
plt.show()

print("Percentil:", p, "Umbral:", t)
print("Blancos prelim:", np.mean(mask_pre == 255))
print("Blancos refinada:", np.mean(mask == 255))
print("Blancos final:", np.mean(mask_final == 255))
print("Guardado: mask_final.png")
```



```
Percentil: 94 Umbral: 183.0
Blancos prelim: 0.06421279907226562
Blancos refinada: 0.16421890258789062
Blancos final: 0.01389694213867185
Guardado: mask_final.png
```

Trade-offs del pipeline (Fourier + Morfología)

La solución propuesta suprime la textura usando un filtro pasa-bajas en Fourier y luego segmenta la anomalía mediante umbralizado y refinamiento morfológico. El principal trade-off es que, al eliminar altas frecuencias para reducir el "tejido", también se atenúan detalles finos: si la rasgadura es muy pequeña o delgada, puede desaparecer o quedar fragmentada y no superar el umbral. Además, el desempeño depende del tipo de tela: cambios en el patrón, el tamaño del grano o la iluminación alteran el espectro y obligan a reajustar parámetros (sigma, percentil y kernels). Por ello, el método es parcialmente robusto, pero tiende a ser específico a condiciones e imagen.